

Merge most of Networking TS into C++ Working Draft

Doc. No.: D1259R1

Date: 2018-11-04

Reply to: Detlef Vollmann, dv@vollmann.ch

Audience: SG1, LEWG, WG21

Merge most of Networking TS into C++ Working Draft

Introduction

The Networking TS grew out of Boost::ASIO and has got a lot of usage experience as part of Boost and in standalone implementations of the TS. Most of the Networking TS is uncontroversial.

One specific part of the Networking TS however clashes with the executor work in SG1. The executor work from SG1 is not yet ready to be merged into the C++ working paper. So this part of the Networking TS should not be merged. But most of the Networking TS can work without an explicitly specified executor model. This paper proposes to merge the executor independent parts of the Networking TS into the C++ working paper and solve any open issues on the Networking TS inside the C++ working paper.

Functionality to be Merged

Without the executor parts not all of the functionality of the TS will be available from the specification in the C++ WD.

This section tries to provide an overview of what functionality is merged and what is left out.

Summary

The Networking TS can be viewed as three separate parts:

- the actual networking, consisting e.g. of the sockets and IP protocol,
- a model for asynchronous operations based on continuations, and
- an interface for executors to run the continuations on.

This proposal merges all of the actual networking part, most of the continuation part (not `uses_future`) and nothing of the executor part.

Defining own execution contexts (based on `io_context`) is not available with the proposed subset and using `strand` is also not possible, but all of the networking operations (synchronous and asynchronous) are merged.

Details

I have myself not worked with any implementation of the Networking TS but with `boost::asio` and Chris Kohlhoff's ASIO implementation available at <http://github.com/chriskohlhoff/asio> (because they provide useful additional functionality for Posix systems). Of the real applications (not toy projects to try some features) most would be possible with the proposed subset, but not all of them.

But to look at the features in more detail, I'll use the examples from Chris in his [github repo](#).

get_executor

When looking through Chris' examples most of them work completely within the subset to be merged. Of those examples that don't work without change most would fail because the use of `get_executor()`. Some of the calls to `get_executor()` are probably just out of habit (e.g. in the constructor of `server` in `cpp03/porthopper/server.cpp`), others are to avoid storing a reference or pointer to the `io_context` in an object if it's already stored inside of e.g. an acceptor subobject anyway. These examples could simply be rewritten not to use `get_executor()` without change in functionality (though at the cost of an additional pointer inside the objects).

Calls that can't be rewritten without affecting the functionality are the cases where there actually are multiple executors and it's important to pick the right one. Probably the only important case here is `strand`.

strand

The Networking TS provides a class `strand` to serialize specific I/O operations based on `io_context` if `io_context` itself uses multiple threads.

`strand` is used in several examples. One of them is a multithreaded HTTP server in `cpp03/http/server3`. (Please note that the actual interface for `strand` differ between Chris' implementation and the Networking TS, but the semantics are the same).

`strand` is an executor and as such not part of the proposed subset.

post

As being part of the main executor interface, `post()` is also not part of the proposed subset.

From a network perspective `post()` can be used to execute arbitrary functions asynchronously. One example that uses this is `cpp03/chat/chat_client.cpp`, which uses `post()` to put the actual function invocation in the queue at a more appropriate place. The example could be simply rewritten to not use `post()` without loss of functionality.

Another example is `cpp03/serialization/connection.hpp`, where `post()` is used to raise an error asynchronously.

Own executor

The only example that uses a custom executor in a networking environment is `invocation/prioritised_handlers.cpp`. This is an example for which the executor interface of the Networking TS exists and therefore is not part of the proposed subset.

Breakdown

Most examples just work, for others (`get_executor()`) some modifications are required. But of course, some functionality is not possible with the proposed subset.

The purpose of this proposal is to provide as much functionality as possible without interfering with any future executor interface in C++. `post()` and `get_executor()` in a minimal version would probably not cause any trouble for future executors.

`strand` in its current form definitely interferes. While it's a convenient mechanism to make network applications scalable on parallel hardware, essentially the same effect can be implemented manually. Should a convenient synchronization mechanism be required now, something like a stripped down version of `strand` inside `io_context` might be possible without having too much inconsistency in future versions of C++.

Wording Overview

The proposal is to merge the Networking TS into the C++ working paper with the following changes:

- Move the contents of the namespace `std::experimental` to namespace `std`.
- Rename the headers `<experimental/*>` to `<*>`.
- From table 3 remove `ExecutionContext` and `Executor`.
- From header `<netfwd>` ([`fwd.decl.synop`]) remove:
 - `executor_binder`
 - `executor_work_guard`
 - `system_executor`
 - `executor`
 - `strand`.
- From header `<executor>` ([`async`]) remove all but:
 - `async_result`, but remove specializations with `use_future_t` and `packaged_task`
 - `async_completion`
 - `associated_allocator`
 - `associated_allocator_t`
 - `get_associated_allocator`
 - `fork_event`
 - `uses_allocator`.
 - Also remove the respective specifications.
- From [`async.reqmts`] remove:
 - [`async.reqmts.executor`]
 - [`async.reqmts.executioncontext`]
 - [`async.reqmts.service`]
 - [`async.reqmts.async.assoc.exec`]
 - [`async.reqmts.async.io.exec`]
 - [`async.reqmts.async.handler.exec`]
 - [`async.reqmts.async.work`]
 - [`async.reqmts.async.completion`]
 - [`async.reqmts.async.exceptions`]
 - [`async.reqmts.async.composed`].
- From [`async.reqmts.async`]p1 remove `Executor1`, `Executor2`, `ex1`, `ex2`, `work1` and `work2`.
- From class `io_context` ([`io__context`]) remove:
 - the public base class `execution_context`
 - `executor_type`
 - `get_executor`
 - paragraphs 1, 2, 4.1 and 4.2.

- In class `io_context` ([`io__context`]) change paragraph 6 remove `get_executor`, and “, and the `io_context::executor_type` copy constructors, member functions and comparison operators,”.
- Remove [`io__context.exec`].
- From `basic_waitable_timer` ([`timer.waitable`]),
`basic_socket` ([`socket.basic`]),
`basic_socket_acceptor` ([`socket.acceptor`]),
and `ip::basic_resolver` ([`internet.resolver`]) remove:
 - `executor_type`
 - `get_executor`
 - from the constructors and assignment the postcondition about `get_executor`
- From table 17 ([`buffer.stream.reqmts.asyncreadstream`]) and table 19 ([`buffer.stream.reqmts.asyncwritestream`]) remove the `get_executor` requirement.
- In `basic_socket_acceptor` ([`socket.acceptor`]) in the accept functions that don't take an `io_context&` as argument replace in the Returns clause `get_executor().context()` with `io_context()`.